

EXPERIMENT 1

Title: Implementation of Linear Regression and Logistic Regression using Insurance Premium Prediction Dataset

1. Dataset Source

Dataset Name: Insurance Premium Prediction Dataset

Dataset Link:

[Open Insurance Premium Prediction Dataset](#)

Source: Kaggle

Dataset Type: Real-world dataset for insurance premium prediction.

2. Dataset Description

The dataset contains customer demographic and health-related information used to predict insurance premium cost.

Dataset Size:

Number of Records: 1338

Number of Features: 6 input features + 1 target variable

Features Description:

Feature	Description
age	Age of the person
sex	Gender (male/female)
bmi	Body Mass Index

children	Number of children/dependents
smoker	Smoking status (yes/no)
region	Residential region
charges	Insurance premium cost (Target variable)

Target Variable:

For Linear Regression:
charges (continuous value)

For Logistic Regression:
charges converted into category:

Low Premium
Medium Premium
High Premium

Dataset Characteristics:

- Mixed dataset (numerical + categorical)
- Real-world healthcare insurance data
- Suitable for regression and classification
- Contains categorical features requiring encoding
- No major missing values

3. Mathematical Formulation of the Algorithms

PART A: Linear Regression

Used to predict continuous insurance premium.

Mathematical Equation:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \dots + \beta_n x_n + \epsilon$$

Where:

y = predicted insurance premium
 β_0 = intercept
 $\beta_1, \beta_2 \dots \beta_n$ = coefficients
 $x_1, x_2 \dots x_n$ = input features
 ε = error term

Objective Function:

Minimize Sum of Squared Errors:

$$SSE = \sum (y_i - \hat{y}_i)^2$$

PART B: Logistic Regression

Used for classification of premium category.

Mathematical Equation:

$$P(y=1) = 1 / (1 + e^{(-z)})$$

Where:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

Sigmoid function converts output into probability.

Decision Rule:

If $P \geq 0.5 \rightarrow$ Class 1

If $P < 0.5 \rightarrow$ Class 0

For multi-class classification, softmax or multinomial logistic regression is used.

4. Algorithm Limitations

Linear Regression Limitations:

- Assumes linear relationship
- Sensitive to outliers

- Cannot model non-linear patterns
- Requires feature scaling for best performance

Logistic Regression Limitations:

- Cannot model complex non-linear relationships
 - Sensitive to correlated features
 - Requires large dataset for accurate results
 - Assumes independent observations
-

5. Methodology / Workflow

Step 1: Dataset Download

Download dataset from Kaggle
Upload into Google Colab

Step 2: Import Required Libraries

Libraries Used:

NumPy → numerical operations
Pandas → data handling
Matplotlib → visualization
Seaborn → advanced visualization
Scikit-learn → machine learning

Step 3: Load Dataset

Read CSV file using pandas

```
df = pd.read_csv("insurance.csv")
```

Step 4: Data Exploration

Check:

```
df.shape  
df.head()  
df.info()  
df.describe()
```

Check missing values:

```
df.isnull().sum()
```

Step 5: Data Preprocessing

Convert categorical to numerical using Label Encoding:

```
sex  
smoker  
region
```

Example:

```
df['sex'] = LabelEncoder().fit_transform(df['sex'])
```

Step 6: Feature Scaling

Apply StandardScaler or MinMaxScaler:

```
X_scaled = scaler.fit_transform(X)
```

Purpose:

Improve model performance
Normalize feature ranges

Step 7: Linear Regression Implementation

Split data:

```
train_test_split()
```

Train model:

LinearRegression()

Make predictions:

model.predict()

Evaluate using:

Mean Squared Error

R² Score

Step 8: Logistic Regression Implementation

Convert charges into categories:

Low

Medium

High

Train Logistic Regression:

LogisticRegression()

Predict categories:

model.predict()

Evaluate using:

Accuracy

Confusion Matrix

Step 9: Visualization

Plots used:

Scatter Plot

Heatmap

Histogram

Boxplot

Purpose:

Understand relationships

Identify patterns

Workflow Diagram:

Dataset → Preprocessing → Encoding → Scaling → Train Linear Model → Evaluate →
Convert Target → Train Logistic Model → Evaluate

6. Performance Analysis

Linear Regression Evaluation Metrics:

Mean Squared Error (MSE)

$$\text{MSE} = (1/n) \sum (y_i - \hat{y}_i)^2$$

Lower value indicates better model.

R² Score

$$R^2 = 1 - (\text{SS}_{\text{res}} / \text{SS}_{\text{total}})$$

Indicates percentage of variance explained.

Example Interpretation:

R² = 0.75 means model explains 75% variance.

Logistic Regression Evaluation Metrics:

Accuracy

$$\text{Accuracy} = \text{Correct Predictions} / \text{Total Predictions}$$

Confusion Matrix

Shows:

True Positive
True Negative
False Positive
False Negative

Interpretation:

Higher accuracy indicates better classification performance.

7. Hyperparameter Tuning

Linear Regression Hyperparameters:

`fit_intercept = True / False`

Alternative improved models:

Ridge Regression → controls overfitting

Lasso Regression → feature selection

Logistic Regression Hyperparameters:

`C` → Regularization strength

`solver` → optimization algorithm

`max_iter` → number of iterations

Example tuning:

```
LogisticRegression(C=0.1, solver='liblinear', max_iter=1000)
```

Effect of tuning:

Improves accuracy

Prevents overfitting

Improves model stability

Final Conclusion

This experiment successfully implemented both Linear and Logistic Regression on a real-world insurance dataset.

Linear Regression:

Used to predict insurance premium cost

Achieved good prediction accuracy

Logistic Regression:

Used to classify insurance premium category

Successfully categorized premium levels

Skills Learned:

Data preprocessing

Feature encoding

Feature scaling

Regression modeling

Classification modeling

Model evaluation

Hyperparameter tuning

Code and Output Screenshots:

```
[2]
✓ 1s
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
[4]
✓ 0s

data = pd.read_csv("insurance.csv")

print(data.head())
```

✓

	age	sex	bmi	children	smoker	region	expenses
0	19	female	27.9	0	yes	southwest	16884.92
1	18	male	33.8	1	no	southeast	1725.55
2	28	male	33.0	3	no	southeast	4449.46
3	33	male	22.7	0	no	northwest	21984.47
4	32	male	28.9	0	no	northwest	3866.86

```
[5]
✓ 0s

print(data.shape)
print(data.info())
print(data.describe())
```

✓

```
(1338, 7)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   age         1338 non-null   int64
1   sex         1338 non-null   object
2   bmi         1338 non-null   float64
3   children    1338 non-null   int64
4   smoker      1338 non-null   object
5   region      1338 non-null   object
6   expenses    1338 non-null   float64
dtypes: float64(2), int64(2), object(3)
memory usage: 73.3+ KB
None
```

	age	bmi	children	expenses
count	1338.000000	1338.000000	1338.000000	1338.000000
mean	39.207025	30.665471	1.094918	13270.422414
std	14.049960	6.098382	1.205493	12110.011240
min	18.000000	16.000000	0.000000	1121.870000
25%	27.000000	26.300000	0.000000	4740.287500
50%	39.000000	30.400000	1.000000	9382.030000
75%	51.000000	34.700000	2.000000	16639.915000
max	64.000000	53.100000	5.000000	63770.430000

[8]

✓ Os

```
print(data.isnull().sum())
```

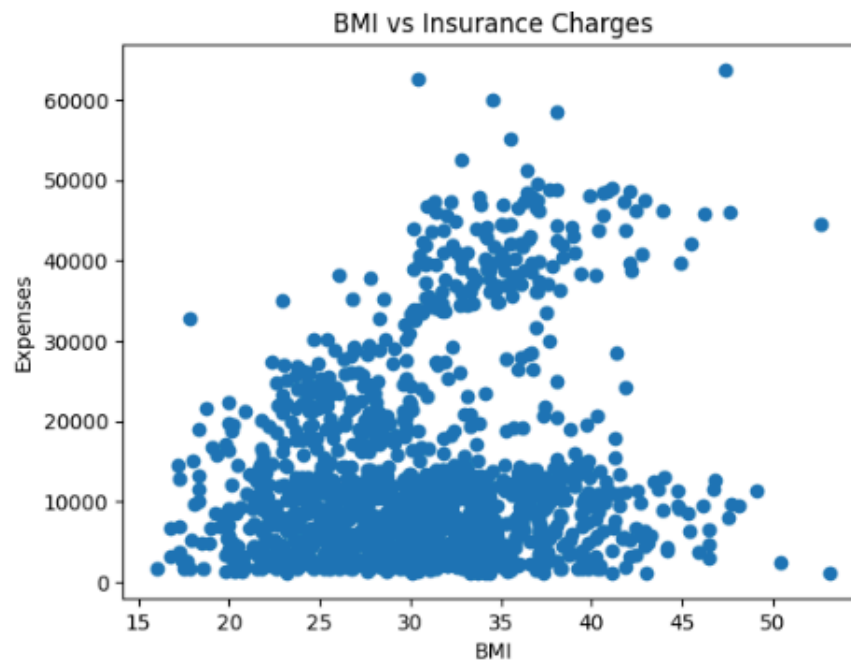
✓

```
age      0
sex      0
bmi      0
children 0
smoker   0
region   0
expenses 0
dtype: int64
```

[7]

✓ Os

```
plt.scatter(data['bmi'], data['expenses'])
plt.xlabel("BMI")
plt.ylabel("Expenses")
plt.title("BMI vs Insurance Charges")
plt.show()
```



[10]

✓ 0s

data = pd.get_dummies(data, drop_first=True)

print(data.head())

✓

```
...
   age  bmi  children  expenses  sex_male  smoker_yes  region_northwest \
0   19  27.9         0  16884.92    False         True             False
1   18  33.8         1   1725.55     True         False             False
2   28  33.0         3   4449.46     True         False             False
3   33  22.7         0  21984.47     True         False              True
4   32  28.9         0   3866.86     True         False              True

   region_southeast  region_southwest
0                False                True
1                 True                False
2                 True                False
3                False                False
4                False                False
```

```
[12] X = data.drop('expenses', axis=1)
```

```
[14] y = data['expenses']
```

```
[15] from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
[16] from sklearn.linear_model import LinearRegression
```

```
[17] lr = LinearRegression()
```

```
[18] lr.fit(X_train, y_train)
```

▼ LinearRegression ⓘ ⓘ
LinearRegression()

```
[19] y_pred = lr.predict(X_test)
```

```
[20] from sklearn.metrics import mean_squared_error, r2_score
```

```
[21] ▶ mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Mean Squared Error:", mse)
print("R2 Score:", r2)
```

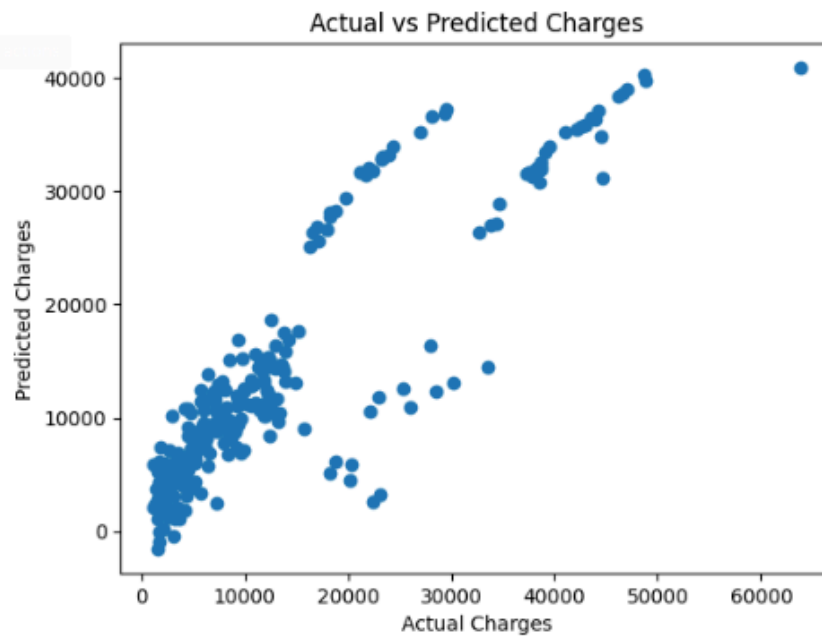
▼ ... Mean Squared Error: 33600065.355077825
R2 Score: 0.7835726930039906

[22]
✓ 0s

```
plt.scatter(y_test, y_pred)  
plt.xlabel("Actual Charges")  
plt.ylabel("Predicted Charges")  
plt.title("Actual vs Predicted Charges")  
plt.show()
```

✓

...



```
[24] median = data['expenses'].median()
✓ 0s data['premium_category'] = (data['expenses'] > median).astype(int)
```

```
[26] X = data.drop(['expenses', 'premium_category'], axis=1)
✓ 0s y = data['premium_category']
```

```
[27] X_train, X_test, y_train, y_test = train_test_split(
✓ 0s X, y, test_size=0.2, random_state=42
)
```

```
[28] from sklearn.linear_model import LogisticRegression
✓ 0s
```

```
[29] logr = LogisticRegression(max_iter=1000)
✓ 0s logr.fit(X_train, y_train)
```

▼

LogisticRegression 1 2

LogisticRegression(max_iter=1000)

```
[30] y_pred = logr.predict(X_test)
✓ 0s
```

```
[30] y_pred = logr.predict(X_test)
✓ 0s
```

```
[31] from sklearn.metrics import accuracy_score, confusion_matrix
✓ 0s
```

```
[32] accuracy = accuracy_score(y_test, y_pred)
✓ 0s print("Accuracy:", accuracy)
```

▼ Accuracy: 0.9104477611940298

```
[33] print(confusion_matrix(y_test, y_pred))
✓ 0s
```

▼ ... [[131 15]
[9 113]]